

# Introduction à Git et Github

Matthieu Vanhoecke

April 28, 2026

## Contents

|            |                                                                          |           |
|------------|--------------------------------------------------------------------------|-----------|
| <b>I</b>   | <b>Commandes de terminal</b>                                             | <b>2</b>  |
| 1          | Système de contrôle de versions (VCS)                                    | 3         |
| <b>II</b>  | <b>Git</b>                                                               | <b>3</b>  |
| 1          | Premiers pas avec Git                                                    | 4         |
|            | Configuration initiale                                                   | 4         |
|            | Création ou récupération d'un dépôt                                      | 4         |
|            | États des fichiers dans Git                                              | 4         |
|            | Suivi et ajout des fichiers                                              | 4         |
|            | Visualisation des modifications                                          | 5         |
|            | Validation des modifications (commit)                                    | 5         |
| 2          | Interactions avancées avec Git                                           | 5         |
|            | Historique des commits                                                   | 5         |
|            | Renommer et supprimer des fichiers                                       | 6         |
|            | Revenir sur des changements et restaurer des fichiers                    | 6         |
|            | Modification d'un commit déjà validé                                     | 6         |
|            | Rollbacks et restauration de commits                                     | 6         |
| 3          | Création et Fusion de Branches                                           | 7         |
|            | Qu'est-ce qu'une branche                                                 | 7         |
|            | Créer de nouvelles branches                                              | 7         |
|            | Visualiser les fichiers dans la branche active                           | 7         |
|            | Interprétation des messages de branche et de HEAD                        | 8         |
|            | Suppression d'une branche                                                | 8         |
|            | Fusion de branches ( <i>merging</i> )                                    | 8         |
|            | Gestion des conflits de fusion                                           | 9         |
|            | Algorithme utilisé lors d'une fusion de branches divergentes             | 9         |
| <b>III</b> | <b>GitHub</b>                                                            | <b>10</b> |
| 1          | Introduction à GitHub                                                    | 10        |
|            | Interactions basiques avec GitHub                                        | 10        |
| 2          | Utilisation d'un dépôt distant                                           | 10        |
|            | Branches locales et distantes après un clone                             | 11        |
|            | Mise à jour d'un dépôt local                                             | 11        |
|            | Mise à jour de tous les dépôts distants : <code>git remote update</code> | 12        |
| 3          | Secure Shell (SSH)                                                       | 12        |
|            | Définition et objectifs                                                  | 12        |
|            | Notions fondamentales : <i>shell</i> et protocole                        | 12        |
|            | Principe de fonctionnement de SSH                                        | 12        |
|            | Méthodes d'authentification                                              | 13        |
|            | Utilisation de SSH                                                       | 13        |
|            | Fonctionnalités avancées de SSH                                          | 13        |
|            | Configuration du serveur SSH                                             | 13        |
|            | Bonnes pratiques de sécurité                                             | 14        |
|            | Synthèse                                                                 | 14        |

|                         |                                                                        |           |
|-------------------------|------------------------------------------------------------------------|-----------|
| 4                       | API Keys . . . . .                                                     | 14        |
|                         | Définition . . . . .                                                   | 14        |
|                         | Authentification et autorisation . . . . .                             | 14        |
|                         | Fonctionnement général . . . . .                                       | 14        |
|                         | Cas d'usage . . . . .                                                  | 14        |
|                         | Limites . . . . .                                                      | 15        |
|                         | Bonnes pratiques de sécurité . . . . .                                 | 15        |
|                         | Synthèse . . . . .                                                     | 15        |
| 5                       | Résolution des conflits . . . . .                                      | 15        |
|                         | Le Pull-Merge-Push . . . . .                                           | 15        |
|                         | Push des branches vers le dépôt distant . . . . .                      | 16        |
|                         | Rebaser ses modifications . . . . .                                    | 16        |
|                         | Bonnes pratiques pour la collaboration . . . . .                       | 16        |
| <b>IV Collaboration</b> |                                                                        | <b>17</b> |
| 1                       | Pull Requests . . . . .                                                | 17        |
|                         | Une pull request simple sur GitHub — via l'interface web . . . . .     | 17        |
|                         | Le workflow typique d'une pull request sur GitHub — en local . . . . . | 17        |
|                         | Mettre à jour une pull request existante . . . . .                     | 17        |
|                         | Fusionner et combiner les commits (Squash) . . . . .                   | 18        |
| 2                       | Code Reviews . . . . .                                                 | 18        |
|                         | The code review workflow . . . . .                                     | 19        |
|                         | Comment utiliser le code reviews . . . . .                             | 19        |
| 3                       | Managing Projects . . . . .                                            | 19        |
|                         | Tracking issues . . . . .                                              | 19        |
|                         | Intégration Continue et Déploiement Continu (CI/CD) . . . . .          | 20        |
| 4                       | Glossary terms . . . . .                                               | 21        |

## I Commandes de terminal

Cette section présente les principales commandes du terminal permettant de comparer et de gérer différentes versions d'un code source (Python, C++, *etc.*). Ces outils sont particulièrement utiles pour analyser les modifications apportées à un ou plusieurs fichiers au cours du développement.

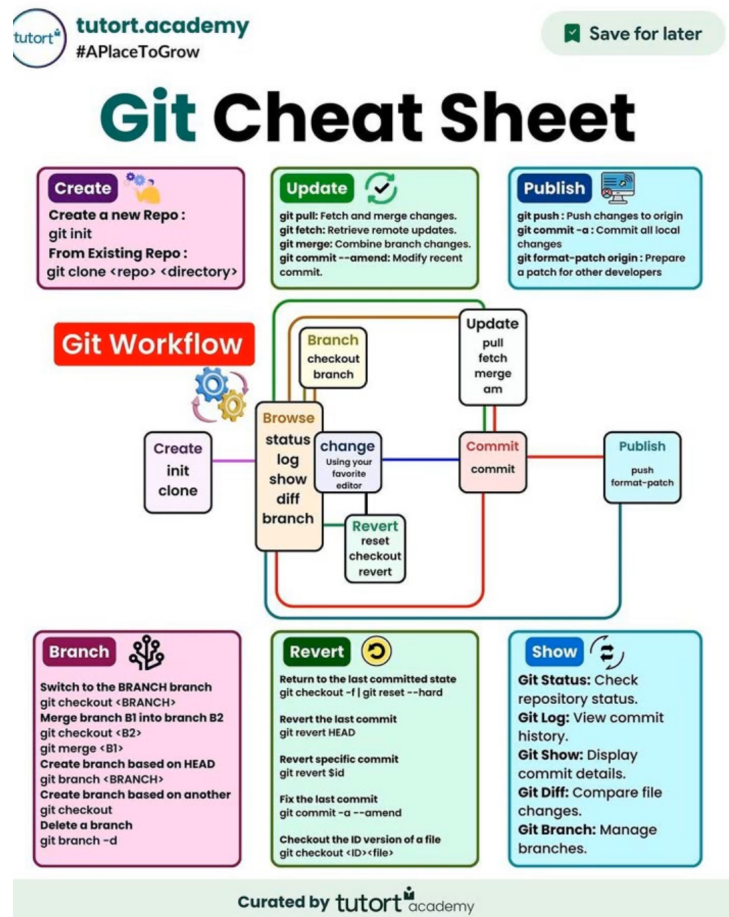
- `diff Version1.py Version2.py` : affiche les différences ligne par ligne entre les fichiers `Version1.py` et `Version2.py`. Le symbole `<` indique une ligne présente uniquement dans le premier fichier, tandis que le symbole `>` indique une ligne présente uniquement dans le second fichier.
- `diff -u Version1.py Version2.py` : génère une sortie au format *unifié*, plus lisible et plus détaillée. Ce format met en évidence le contexte des modifications (par exemple à l'intérieur des boucles, des conditions `if` ou des fonctions), ce qui facilite l'analyse et la compréhension des changements.
- `wdiff Version1.py Version2.py` : compare les fichiers mot à mot plutôt que ligne par ligne. Contrairement à `diff`, qui affiche des lignes entières modifiées, `wdiff` permet d'identifier précisément les mots ajoutés ou supprimés. Cette commande est particulièrement adaptée aux fichiers textuels ou à la documentation.
- `diff -u oldfile newfile > change.diff` : le symbole `>` redirige la sortie de la commande `diff` vers le fichier `change.diff`. Cette commande permet de générer un fichier recensant l'ensemble des modifications entre deux versions. Ce fichier est appelé un *fichier diff* ou *patch*.
- `patch fichierpython.py < change.diff` : applique au fichier `fichierpython.py` les modifications décrites dans le fichier `change.diff`. Le symbole `<` indique que le contenu du fichier `change.diff` est utilisé comme entrée de la commande `patch`. Cette méthode fonctionne également sur des répertoires entiers : il est possible de générer un diff sur un ensemble de fichiers, puis d'appliquer les modifications simultanément à un projet contenant des dizaines, voire des centaines de fichiers.
- `ls -la` : affiche tout les fichiers commençant pour un point "."

## 1 Système de contrôle de versions (VCS)

Un *Version Control System* (VCS), ou système de contrôle de versions, est un outil permettant de gérer l'évolution de fichiers au cours du temps.

- Il permet de stocker du code source, des fichiers de documentation et leurs différentes versions.
- Il assure le suivi des modifications apportées aux fichiers, en conservant des informations telles que la date, l'heure et l'auteur des changements.
- Les fichiers sont généralement organisés au sein de **repositories** (dépôts), qui regroupent le code source, la documentation et les fichiers annexes.
- Il stocke également les configurations nécessaires au bon fonctionnement des projets.
- Un *commit* est un ensemble cohérent de modifications soumis au système de contrôle de versions afin d'être conservé de manière fiable et traçable.

## II Git



Git est un système de contrôle de versions *open source*. Il repose sur une architecture distribuée : chaque contributeur d'un dépôt possède une copie complète de l'historique du projet sur sa machine de développement. Git peut fonctionner à la fois comme client et comme serveur, et son utilisation ne nécessite pas obligatoirement une connexion à Internet. Il est donc possible de travailler localement, puis de synchroniser les modifications ultérieurement.

- Pour connaître la version de Git installée : `git --version`

## 1 Premiers pas avec Git

Cette sous-section présente les commandes essentielles pour débiter avec Git, ainsi que les concepts fondamentaux nécessaires à sa bonne utilisation.

### Configuration initiale

Avant toute utilisation, il est recommandé de configurer Git avec des informations personnelles. Ces informations seront associées à chaque *commit*.

- `git config --global user.email "mail@exemple.com"`
- `git config --global user.name "Mon Nom"`

L'option `--global` indique que ces paramètres s'appliquent à tous les dépôts Git de l'utilisateur sur la machine. Pour afficher l'ensemble des paramètres de configuration courants, on peut utiliser :

- `git config -l`

### Création ou récupération d'un dépôt

Deux méthodes permettent de commencer à travailler avec Git :

- créer un nouveau dépôt local ;
- cloner un dépôt existant depuis un serveur distant (abordé ultérieurement).

La création d'un nouveau dépôt s'effectue avec la commande :

- `git init`

Cette commande initialise un dépôt Git vide dans le répertoire courant. Le répertoire Git (dossier `.git`) contient l'ensemble des modifications et leur historique, tandis que l'arborescence de travail (*Working Tree*) contient les versions actuelles des fichiers du projet.

### États des fichiers dans Git

Dans Git, un fichier peut se trouver dans l'un des trois états suivants :

- **Modifié** : le fichier a été modifié, mais les changements ne sont pas encore suivis par Git.
- **Staged** (zone de transit) : les modifications sont prêtes à être validées.
- **Commit** : les modifications sont enregistrées de manière permanente dans l'historique du dépôt.

Le cycle de base est donc :

*modifié* → *staged* → *commit*

### Suivi et ajout des fichiers

Pour que Git commence à suivre un fichier (par exemple `exemple.py`), celui-ci doit se trouver dans l'arborescence de travail, puis être ajouté à la zone de transit :

- `git add exemple.py`

on peut utiliser aussi

- `git add *`

pour ajouter tout les fichiers/changement pas dans la zone de transit. La commande suivante permet de consulter l'état du dépôt à tout moment :

- `git status`

Elle indique quels fichiers sont modifiés, suivis, en attente de validation ou déjà enregistrés.

## Visualisation des modifications

Git fournit plusieurs commandes pour examiner les changements effectués :

- `git diff` : affiche les différences entre les fichiers modifiés et leur dernière version validée (*commit*). Autrement dit, cette commande montre les changements qui ne sont pas encore dans la zone de transit.
- `git diff --staged` : affiche les différences entre les fichiers présents dans la zone de transit et le dernier *commit*. Elle permet donc de vérifier précisément ce qui sera enregistré lors du prochain *commit*.

## Validation des modifications (commit)

Une fois les fichiers ajoutés à la zone de transit, il est nécessaire de valider les modifications à l'aide d'un *commit* :

- `git commit`

Cette commande ouvre un éditeur de texte dans lequel l'utilisateur doit saisir un message de *commit*. Pour écrire directement le message depuis le terminal, on peut utiliser :

- `git commit -m "Message du commit"`

Un bon message de *commit* se compose :

- d'une courte description de la modification (environ 50 caractères) ;
- éventuellement suivie d'une ligne vide et d'un texte plus détaillé expliquant le changement.

Il est également possible de valider directement les fichiers déjà suivis, sans passer explicitement par `git add` :

- `git commit -a -m "Message du commit"`

Cette commande ne fonctionne que pour les fichiers déjà ajoutés au dépôt par le passé. Lors de l'ajout initial d'un fichier, l'utilisation de `git add` est obligatoire.

## 2 Interactions avancées avec Git

### Historique des commits

Pour consulter l'historique des validations (*commits*) effectuées dans un dépôt, Git met à disposition plusieurs commandes permettant d'obtenir différents niveaux de détail :

- `git log` : affiche la liste des commits, incluant leur identifiant (hash), la date, l'auteur et le message associé.
- `git log -p` : affiche, en complément, les modifications détaillées (*patch*) introduites par chaque commit.
- `git log -n` : affiche uniquement les *n* derniers commits, où *n* est un entier strictement positif.
- `git log --stat` : présente un résumé des changements, indiquant les fichiers modifiés ainsi que le nombre de lignes ajoutées et supprimées.
- `git log --graph --oneline` : pour voir les commits comme un graphes et en une seule ligne par commit.
- `git show <ID_commit>` : affiche les informations détaillées et les changements associés à un commit spécifique.

On utilise `git show` sans l'ID du commit afin de voir les informations concernant le dernier commit.

## Renommer et supprimer des fichiers

Git permet de gérer explicitement le renommage et la suppression de fichiers tout en assurant le suivi de ces opérations dans l'historique du dépôt :

- `git rm code.py` : supprime le fichier `code.py` du répertoire de travail et marque cette suppression pour le prochain *commit*.
- `git mv oldname.py newname.py` : renomme le fichier `oldname.py` en `newname.py`. Cette commande équivaut à un renommage suivi automatiquement par Git.

Toute suppression ou modification du nom d'un fichier doit être validée par un *commit* afin d'être enregistrée dans l'historique du dépôt, par exemple :

- `git commit -m 'Suppression de l'ancien fichier'`

Il est également possible d'indiquer à Git d'ignorer certains fichiers (fichiers temporaires, sorties de programme, fichiers générés automatiquement, etc.) à l'aide du fichier `.gitignore`. Les fichiers listés dans ce fichier ne seront pas suivis par Git.

## Revenir sur des changements et restaurer des fichiers

Pour annuler les modifications locales effectuées sur un fichier suivi (par exemple `exemple.py`) et le restaurer dans l'état correspondant au dernier *commit*, la commande suivante peut être utilisée :

- `git checkout NomDuFichier`

Cette commande restaure le fichier tel qu'il était lors de la dernière validation, annulant ainsi toutes les modifications locales non validées.

Si un fichier a déjà été ajouté à la zone de transit (*staging area*) à l'aide de `git add`, il est possible de le retirer de cette zone avec la commande :

- `git reset HEAD NomDuFichier`

Cette commande permet de faire revenir le fichier de l'état *staged* à l'état *modifié*, sans perdre les changements effectués.

Pour ces commandes, il est possible d'utiliser le flag :

- `-p`

Ce mode interactif permet de sélectionner précisément les parties des modifications à annuler ou à conserver, sans nécessairement revenir en arrière sur l'ensemble des changements.

## Modification d'un commit déjà validé

Afin de modifier le dernier commit réalisé (ajout de fichiers oubliés, correction du message de commit, etc.), il est possible d'utiliser la commande suivante :

- `git commit --amend`

Cette commande crée un nouveau commit en écrasant le commit précédent. Elle permet d'ajouter ou de modifier des fichiers, ainsi que de changer le message du commit, tout en supprimant l'ancien commit de l'historique.

Cette fonctionnalité est particulièrement utile pour corriger rapidement une erreur ou pour inclure des fichiers oubliés. Toutefois, il est fortement déconseillé d'utiliser cette commande dans un dépôt public ou collaboratif, car elle modifie l'historique et peut entraîner des incohérences pour les autres utilisateurs.

## Rollbacks et restauration de commits

Il est possible d'annuler les effets d'un commit après sa validation en utilisant la commande :

- `git revert HEAD`

Cette commande crée un nouveau commit contenant l'inverse exact des changements introduits par le commit ciblé. L'alias `HEAD` désigne ici le dernier commit effectué.

Pour annuler un commit plus ancien, il est possible de spécifier explicitement son identifiant :

- `git revert <ID_commit>`

L'identifiant d'un commit est un hash cryptographique calculé à partir de l'ensemble des informations du commit, garantissant la cohérence de l'historique, notamment dans les environnements collaboratifs. En pratique, seuls les 4 à 6 premiers caractères de cet identifiant suffisent généralement pour que Git retrouve le commit correspondant.

### 3 Création et Fusion de Branches

#### Qu'est-ce qu'une branche

Une *branche* (*branch*) est un pointeur vers un commit particulier. Plus précisément, elle représente une ligne de développement indépendante au sein d'un projet. Le commit vers lequel une branche pointe correspond au dernier maillon d'une chaîne de commits, c'est-à-dire à l'état le plus récent de cette ligne de développement.

Par défaut, lorsqu'un nouveau dépôt Git est initialisé, Git crée automatiquement une branche principale, traditionnellement nommée **master** (ou **main** dans les conventions récentes). Cette branche contient l'historique principal et stable du projet.

L'un des principaux intérêts des branches réside dans leur capacité à isoler le développement. Lorsqu'un développeur souhaite implémenter une nouvelle fonctionnalité, corriger un bug ou expérimenter une idée, il peut créer une branche distincte afin de travailler sans affecter la branche principale. Une fois le développement terminé et validé, il est possible de fusionner (*merge*) cette branche avec la branche principale, intégrant ainsi les changements de manière contrôlée. Les branches constituent ainsi un mécanisme fondamental de Git, facilitant le travail collaboratif, l'expérimentation et la gestion parallèle de plusieurs évolutions du projet.

#### Créer de nouvelles branches

La commande `git branch` permet de lister, créer, supprimer et manipuler les branches d'un dépôt Git. En particulier, les commandes suivantes sont couramment utilisées :

- `git branch` : affiche la liste des branches du dépôt.
- `git branch nomBranche` : crée une nouvelle branche nommée `nomBranche`.

Lors de l'affichage des branches avec `git branch`, un astérisque (\*) indique la branche actuellement active, c'est-à-dire celle sur laquelle le travail est en cours.

Pour basculer vers une autre branche existante, on utilise la commande :

- `git checkout nomBranche`

Il est également possible de créer une nouvelle branche et de s'y positionner immédiatement à l'aide d'une seule commande :

- `git checkout -b nomBranche`

Cette commande crée la branche `nomBranche` et met à jour l'espace de travail pour pointer vers celle-ci.

Après la création d'une branche, le cycle de travail reste identique : il est nécessaire d'ajouter les fichiers modifiés à la zone de transit à l'aide de `git add`, puis de valider les changements avec un *commit*. Les commits réalisés sur cette branche n'affectent pas les autres branches tant qu'aucune fusion (*merge*) n'est effectuée.

#### Visualiser les fichiers dans la branche active

La commande `ls -l` permet d'afficher tous les fichiers présents dans le répertoire de travail. Concrètement, elle montre les fichiers correspondant à la `**branche actuellement active**`, c'est-à-dire celle sur laquelle **HEAD** pointe.

- **Remarque** : `ls -l` n'affiche pas les fichiers des autres branches. Pour voir le contenu d'une autre branche, il faut soit changer de branche avec `git checkout`, soit utiliser une commande Git spécifique comme `git ls-tree`.
- Les fichiers listés comprennent à la fois les fichiers suivis et non suivis présents dans le répertoire de travail. Pour lister uniquement les fichiers suivis par Git, on peut utiliser :

– `git ls-files`

## Interprétation des messages de branche et de HEAD

Lors de l'exécution de la commande `git log`, Git affiche non seulement l'historique des commits, mais également des informations sur les branches et la position de HEAD. Ces indications permettent de comprendre rapidement sur quelle branche on travaille et quelles branches pointent vers quels commits.

Dans la sortie de `git log`, on peut observer des annotations telles que :

- **HEAD** : représente la position actuelle de l'utilisateur dans le dépôt. Il pointe généralement vers la branche active.
- **HEAD -> nomBranche** : indique que HEAD pointe vers la branche **nomBranche**, ce qui signifie que cette branche est actuellement active.
- **nomBranche** (par exemple **master**, **main** ou **new\_branch**) : indique qu'une branche pointe vers ce commit précis.

Par exemple, lorsqu'un commit est effectué sur la branche principale, la sortie de `git log` peut contenir une ligne similaire à :

```
HEAD -> master
```

Cela signifie que HEAD et la branche **master** pointent tous deux vers le dernier commit de l'historique.

En revanche, si un commit est réalisé sur une branche secondaire, par exemple **new\_branch**, la sortie pourra indiquer :

```
HEAD -> new_branch
```

Dans ce cas, la branche principale (**master** ou **main**) continuera de pointer vers un commit plus ancien, ce qui reflète le fait que les nouveaux changements sont isolés dans la branche secondaire.

Ces annotations permettent ainsi de visualiser clairement :

- la branche actuellement active,
- les branches qui partagent le même commit,
- les divergences entre différentes lignes de développement.

La compréhension de ces messages est essentielle pour naviguer efficacement dans l'historique d'un dépôt Git et éviter toute confusion lors du travail avec plusieurs branches.

## Suppression d'une branche

Pour supprimer une branche locale, on utilise la commande suivante :

- `git branch -d nomBranche`

Cette commande supprime la branche spécifiée uniquement si tous ses commits ont été fusionnés avec une autre branche. Si vous souhaitez forcer la suppression même si la branche n'a pas été fusionnée, vous pouvez utiliser :

- `git branch -D nomBranche`

## Fusion de branches (*merging*)

La fusion (*merge*) permet d'intégrer les modifications d'une branche dans une autre. Par exemple, pour fusionner une branche secondaire dans la branche principale, procédez comme suit :

1. Basculez sur la branche principale (par exemple **master** ou **main**) :

- `git checkout master`

2. Fusionnez la branche secondaire dans la branche principale :

- `git merge nomBrancheASFusionner`

Git tente automatiquement de combiner les changements ligne par ligne. Si les modifications de la branche principale et de la branche secondaire touchent des parties différentes d'un fichier, la fusion se fera automatiquement.



## Gestion des conflits de fusion

Dans certains cas, Git ne peut pas fusionner automatiquement les branches, généralement lorsque les mêmes lignes d'un fichier ont été modifiées dans les deux branches. Ces situations sont appelées **\*\*conflits de fusion\*\***. Voici comment les gérer :

1. Git marque les fichiers en conflit et insère des délimitations spéciales dans le fichier pour indiquer les modifications de chaque branche :

```
<<<<<< HEAD
contenu de la branche courante
=====
contenu de la branche fusionnée
>>>>>> nomBrancheASFusionner
```

2. Il faut alors éditer manuellement le fichier pour résoudre le conflit, en choisissant ou combinant les changements.
3. Une fois les conflits résolus, ajouter les fichiers à la zone de transit :

- `git add fichier_resolu`

4. Finaliser la fusion avec un commit :

- `git commit`

### Remarques :

- Les conflits sont normaux lorsqu'on travaille en parallèle sur plusieurs branches.
- Git fournit des outils graphiques et en ligne de commande (`git mergetool`) pour faciliter la résolution des conflits.

Pour interrompre la fusion on peut utiliser,

- `git merge --abort`

Cela arrête la fusion et réinitialise les fichiers dans chacune des branches.

## Algorithme utilisé lors d'une fusion de branches divergentes

Lorsque deux branches ont divergé, c'est-à-dire que chacune contient des commits absents de l'autre depuis leur ancêtre commun, Git utilise l'**algorithme de fusion à trois voies** (*three-way merge*) :

- Git identifie le *commit ancêtre commun* (*merge base*) des deux branches.
- Il compare les modifications effectuées sur la branche courante (**HEAD**) et sur la branche à fusionner par rapport à ce commit commun.
- Git tente de combiner automatiquement les changements.
- Si des modifications affectent les mêmes lignes d'un fichier dans les deux branches, un **conflit de fusion** se produit, nécessitant une résolution manuelle.

**Remarque :** Si la branche courante n'a pas de nouveaux commits depuis la branche à fusionner, Git effectue une *fusion rapide* (*fast-forward*) plutôt qu'une fusion à trois voies.

## III GitHub

GitHub est un service d'hébergement de dépôts Git. En plus des fonctionnalités de contrôle de version offertes par Git, GitHub fournit des outils supplémentaires tels que le suivi des bogues (*issue tracking*), les wikis, la gestion de projets et la collaboration entre développeurs.

GitHub permet de partager des dépôts via le Web, de les consulter à distance et de les cloner sur un ordinateur local afin de pouvoir y travailler. D'autres plateformes similaires existent, telles que Bitbucket ou GitLab, qui offrent des fonctionnalités comparables.

Dans un contexte nécessitant un niveau de sécurité plus élevé ou pour la gestion de données sensibles (notamment en entreprise), il est courant de mettre en place un serveur Git privé, hébergé en interne.

### 1 Introduction à GitHub

#### Interactions basiques avec GitHub

Pour créer une copie locale d'un dépôt Git hébergé sur GitHub, on utilise la commande suivante :

- `git clone URL-du-depot`

Cette commande clone le dépôt distant sur la machine locale et permet d'utiliser ensuite toutes les commandes Git sur cette copie.

Une fois des modifications effectuées et validées localement à l'aide de `git commit`, il est possible d'envoyer ces changements vers le dépôt distant avec :

- `git push`

Cette commande transmet tous les commits locaux vers le dépôt GitHub correspondant.

**Authentification** Lors des échanges avec un dépôt distant (envoi ou récupération de données), GitHub demande une authentification. Par défaut, celle-ci peut se faire à l'aide d'un identifiant et d'un mot de passe. Il est cependant recommandé d'utiliser une clé SSH pour éviter une authentification répétée et améliorer la sécurité.

Il est également possible d'utiliser un assistant de gestion des identifiants, par exemple :

- `git config --global credential.helper cache`

Cette configuration met en cache les identifiants pendant une durée limitée (par défaut 15 minutes).

Pour récupérer les nouvelles modifications présentes sur le dépôt distant et les intégrer à la branche locale courante, on utilise :

- `git pull`

Cette commande combine deux opérations : la récupération des nouveaux commits depuis le dépôt distant (`fetch`) et leur intégration dans la branche locale (`merge` ou `rebase`, selon la configuration).

### 2 Utilisation d'un dépôt distant

Lorsqu'un dépôt distant est cloné, Git configure automatiquement ce dépôt sous le nom `origin`. Le terme `origin` est un *alias* désignant l'URL du dépôt distant à partir duquel le projet a été cloné.

#### Signification du terme `origin`

- Représente le dépôt distant principal.
- Permet de référencer facilement le dépôt distant sans retaper l'URL complète.
- Il est possible d'avoir plusieurs dépôts distants, chacun avec un nom différent (par exemple `origin`, `upstream`, etc.).

## Commandes pour interagir avec le dépôt distant

- `git remote -v` : liste les dépôts distants configurés et leurs URL pour *fetch* et *push*.
- `git remote show origin` : fournit des informations détaillées sur les branches suivies et l'état de synchronisation.
- `git branch -r` : affiche uniquement les branches distantes.
- `git status` : indique l'état de la branche locale par rapport à sa branche distante.

## Branches locales et distantes après un clone

Après un clonage, Git configure :

- des **\*\*branches distantes locales\*\***, par exemple `origin/master` : copies locales en lecture seule des branches du dépôt distant, reflétant l'état du dépôt GitHub.
- des **\*\*branches locales\*\***, par exemple `master` : branches de travail modifiables sur lesquelles tu ajoutes des commits. Elles sont suivies par le pointeur `HEAD`.

**Flux de mise à jour et synchronisation** Pour rester à jour avec le dépôt distant et intégrer les changements :

1. `git fetch` : met à jour les branches distantes locales (`origin/...`) sans modifier les branches locales.
2. `git log origin/master` : permet de visualiser l'historique des commits sur la branche distante.
3. `git merge origin/master` ou `git rebase origin/master` : intègre les changements dans la branche locale.
4. `git push` : envoie les commits locaux vers le dépôt distant.

## Résumé pratique

- `origin/master` = reflet du dépôt distant (lecture seule).
- `master` = branche locale modifiable.
- `fetch` et `git remote update` mettent à jour uniquement les branches distantes locales.
- `merge/rebase` intègre les changements distants dans la branche locale.
- `push` envoie les commits locaux vers GitHub.

## Mise à jour d'un dépôt local

Dans un contexte collaboratif, d'autres contributeurs peuvent ajouter des commits au dépôt distant pendant que l'on travaille localement. Ces modifications ne sont pas automatiquement répercutées dans le dépôt local.

## Vérification de l'état des branches locales et distantes

- `git remote show origin` : permet de constater si certaines branches locales sont *obsolètes* par rapport aux branches distantes.

## Récupération des changements

- `git fetch` : récupère les nouveaux commits depuis le dépôt distant sans modifier la branche locale, mettant à jour uniquement les branches distantes locales.
- `git log origin/master` : permet d'examiner les commits récents sur la branche distante.
- `git status` : indique si la branche locale est en retard ou en avance par rapport à sa branche distante.

## Intégration des changements

- `git merge origin/master` : fusionne la branche distante `origin/master` dans la branche locale. Si la branche locale n'a pas divergé, Git effectue une *fusion rapide* (*fast-forward*).

## Flux de travail recommandé

1. utiliser `git fetch` pour examiner les changements sur le dépôt distant,
2. vérifier leur contenu et leur historique,
3. utiliser `git merge` pour intégrer ces changements dans la branche locale.

**Récupération et fusion combinées** La commande `git pull` combine automatiquement `fetch` et `merge` pour mettre à jour la branche locale avec les commits du dépôt distant.

### Mise à jour de tous les dépôts distants : `git remote update`

- `git remote update` : met à jour toutes les références de tous les dépôts distants configurés.
- Contrairement à `git fetch`, qui cible un dépôt ou une branche spécifique, cette commande synchronise toutes les branches distantes pour tous les dépôts distants connus.
- Elle n'affecte pas les branches locales, mais permet d'avoir une vue complète et à jour de l'état des branches distantes avant de faire des fusions ou d'autres opérations.

## 3 Secure Shell (SSH)

### Définition et objectifs

Le *Secure Shell* (SSH) est un protocole réseau sécurisé permettant d'accéder à une machine distante via une interface en ligne de commande. Il fournit un canal de communication chiffré au-dessus d'un réseau non sécurisé, garantissant :

- la confidentialité des données échangées ;
- l'intégrité des communications ;
- l'authentification des machines et des utilisateurs.

SSH est un standard incontournable pour l'administration de serveurs Linux/Unix et pour de nombreuses opérations liées au développement et à la gestion d'infrastructures.

### Notions fondamentales : *shell* et protocole

Un *shell* est un programme servant d'interface entre l'utilisateur et le système d'exploitation, permettant d'exécuter des commandes.

Un *protocole* est un ensemble de règles définissant comment deux systèmes communiquent entre eux. Les protocoles réseau sont généralement définis comme des standards ouverts afin de garantir l'interopérabilité entre différents systèmes.

SSH est donc un protocole fournissant un *shell distant sécurisé*.

### Principe de fonctionnement de SSH

SSH repose sur la cryptographie à clé publique combinée à du chiffrement symétrique :

- lors de la première connexion, le serveur SSH envoie sa **clé publique** au client ;
- le client vérifie l'empreinte de cette clé afin de s'assurer de l'identité du serveur ;
- le client et le serveur négocient ensuite des algorithmes de chiffrement communs ;
- un canal chiffré est établi pour toute la durée de la session.

La **clé privée** du serveur, conservée secrète, est indispensable pour déchiffrer les données. Par défaut, un serveur SSH écoute sur le port réseau 22.

## Méthodes d'authentification

SSH propose principalement deux méthodes d'authentification :

- **Authentification par mot de passe** : simple à mettre en place, mais moins sécurisée ;
- **Authentification par clés SSH** : méthode recommandée, plus sécurisée et adaptée à l'automatisation.

Une clé SSH est composée d'une paire *clé publique* / *clé privée*. La clé publique peut être copiée sur les serveurs distants, tandis que la clé privée doit rester strictement confidentielle.

## Utilisation de SSH

### Connexion à un serveur distant

- `ssh utilisateur@hote` : ouvre une session distante sécurisée ;
- `ssh -p port utilisateur@hote` : se connecte via un port SSH spécifique.

### Génération de clés SSH

- `ssh-keygen` : génère une paire de clés SSH ;
- `ssh-keygen -t rsa -b 2048` : génère une clé RSA de 2048 bits.

Les clés sont stockées par défaut dans le répertoire `~/.ssh/`.

### Installation de la clé publique sur un serveur

- `ssh-copy-id utilisateur@hote` : copie automatiquement la clé publique sur le serveur distant.

### Exécution de commandes à distance

- `ssh utilisateur@hote "commande"` : exécute une commande sans ouvrir de session interactive.

### Transfert de fichiers sécurisé

- `scp fichier utilisateur@hote:/chemin` : copie un fichier vers un serveur distant ;
- `sftp utilisateur@hote` : ouvre une session de transfert de fichiers sécurisée.

## Fonctionnalités avancées de SSH

SSH offre plusieurs fonctionnalités avancées :

- **Port forwarding (tunneling)** : redirection sécurisée de ports réseau ;
- **Jump host / bastion** : accès à des serveurs internes via un serveur intermédiaire ;
- **X11 forwarding** : exécution d'applications graphiques distantes avec affichage local.

Ces fonctionnalités peuvent être configurées via le fichier `~/.ssh/config`.

## Configuration du serveur SSH

Le serveur SSH est fourni par le service `sshd`. Sur les systèmes Linux, son fichier de configuration principal est :

- `/etc/ssh/sshd_config`

Il permet notamment de :

- activer ou désactiver l'authentification par mot de passe ;
- restreindre les utilisateurs autorisés ;
- renforcer la sécurité globale du serveur.

## Bonnes pratiques de sécurité

- Ne jamais partager une clé privée SSH.
- Protéger les clés privées par une phrase secrète.
- Désactiver l'authentification par mot de passe lorsque les clés sont utilisées.
- Vérifier toute alerte indiquant un changement de clé du serveur distant.

## Synthèse

SSH est un outil fondamental pour l'administration sécurisée des systèmes. Grâce au chiffrement, à l'authentification par clés et à ses nombreuses fonctionnalités, il constitue un pilier de la gestion des infrastructures modernes.

## 4 API Keys

### Définition

Une *API Key* est un jeton d'authentification utilisé par une application pour accéder à une interface de programmation (API). Elle permet à un service distant d'identifier le projet ou l'application effectuant une requête et de contrôler les droits associés.

### Authentification et autorisation

Les API keys remplissent deux fonctions principales :

- **Authentification** : vérifier qu'une application est autorisée à appeler l'API ;
- **Autorisation** : définir quelles ressources ou actions sont accessibles.

Contrairement à d'autres mécanismes, une API key identifie généralement une *application* et non un utilisateur individuel.

### Fonctionnement général

Une API key est :

- générée par le fournisseur de l'API ;
- envoyée avec chaque requête ;
- associée à des permissions et des quotas.

Elle peut être transmise :

- via un paramètre d'URL ;
- via un en-tête HTTP (méthode recommandée) ;
- plus rarement via un endpoint dédié.

### Cas d'usage

Les API keys sont adaptées pour :

- bloquer l'accès anonyme à une API ;
- limiter le nombre de requêtes ;
- analyser les usages et statistiques ;
- filtrer les journaux par application.

## Limites

Les API keys ne doivent pas être utilisées pour :

- identifier précisément des utilisateurs ;
- fournir une autorisation fine et hautement sécurisée ;
- protéger seules des données sensibles.

Des solutions comme *OAuth* sont préférables pour ces usages.

## Bonnes pratiques de sécurité

- Ne jamais inclure une API key dans un dépôt public.
- Utiliser des variables d'environnement pour les stocker.
- Restreindre permissions et quotas.
- Régénérer régulièrement les clés compromises.

## Synthèse

Les API keys constituent un mécanisme simple et efficace pour contrôler l'accès aux API. Elles doivent cependant être utilisées avec prudence et combinées à des mécanismes plus robustes lorsque la sécurité est critique.

## 5 Résolution des conflits

### Le Pull-Merge-Push

Dans un environnement collaboratif, il est fréquent que plusieurs personnes modifient le même dépôt en parallèle. Un conflit peut survenir lorsque l'on tente d'envoyer (**push**) des modifications locales alors que de nouveaux commits ont déjà été ajoutés sur le dépôt distant (GitHub) par un autre collaborateur.

**Origine des conflits** Lorsque Git détecte que la branche locale est en retard par rapport à la branche distante, il refuse le **push** afin d'éviter d'écraser le travail des autres. Un message d'erreur indique alors que la branche distante contient des modifications inconnues localement.

**Synchronisation avec le dépôt distant** Dans ce cas, il est nécessaire de synchroniser d'abord le dépôt local avec le dépôt distant en utilisant :

- `git pull`

Cette commande récupère les nouveaux commits du dépôt distant et tente de les fusionner avec la branche locale courante.

**Gestion des conflits de fusion** Si Git ne parvient pas à fusionner automatiquement les modifications (par exemple lorsque les mêmes lignes d'un fichier ont été modifiées), il signale un *conflit de fusion*. Les fichiers concernés sont alors marqués comme conflictuels et doivent être corrigés manuellement par le développeur. Une fois les conflits résolus, il faut :

- ajouter les fichiers corrigés avec `git add`,
- valider la fusion avec `git commit`.

**Visualisation de l'historique des commits** Pour visualiser l'arbre des commits sur l'ensemble des branches (locales et distantes), on peut utiliser :

- `git log --graph --oneline --all`

Cette commande permet de mieux comprendre l'historique du projet et les divergences entre branches.

**Inspection des changements** Pour examiner précisément les modifications introduites par les commits d’une branche distante, on peut utiliser :

- `git log -p origin/master`

Cette commande affiche le détail des changements ligne par ligne, ce qui est utile pour analyser les modifications avant ou après une fusion.

**Résumé** En cas de conflit lors d’un `push`, il est essentiel de :

- synchroniser le dépôt local avec `git pull`,
- résoudre manuellement les conflits éventuels,
- valider la fusion avant de pousser à nouveau les changements vers GitHub.

### Push des branches vers le dépôt distant

Les branches permettent de travailler sur plusieurs versions d’un même projet (par exemple une version stable et une version bêta) ou d’implémenter de nouvelles fonctionnalités sans affecter le code principal. Pour envoyer une branche locale vers un dépôt distant, on utilise la commande suivante :

- `git push -u origin nom_de_la_branche`

où `origin` est le nom du dépôt distant (par défaut, le dépôt principal sur GitHub ou GitLab) et `nom_de_la_branche` est la branche locale que l’on souhaite pousser. L’option `-u` (ou `--set-upstream`) permet de définir la branche distante comme branche amont, ce qui facilite l’utilisation ultérieure des commandes `git push` et `git pull` sans préciser le nom du dépôt et de la branche.

### Rebaser ses modifications

Pour intégrer les modifications d’une branche dans la branche principale, deux méthodes sont couramment utilisées : `git merge` et `git rebase`. La commande `git merge` effectue une fusion classique, tandis que `git rebase` permet de réappliquer les commits d’une branche sur une nouvelle base. La commande suivante permet de rebaser une branche :

- `git rebase nom_branche`

où `nom_branche` est la branche que l’on souhaite utiliser comme nouvelle base (généralement la branche principale). Le rebasing consiste à changer le commit de base d’une branche afin de conserver un historique linéaire. Cette méthode facilite les fusions rapides et rend l’historique du dépôt plus lisible, notamment lors de la correction de bugs ou du développement de fonctionnalités. Après avoir rebasé la branche, on peut effectuer la fusion avec la branche principale à l’aide de la commande :

- `git merge nom_branche_a_fusionner`

Une fois le travail terminé et la fusion effectuée, il est recommandé de supprimer les branches secondaires. Pour supprimer une branche distante :

- `git push --delete origin nom_branche_a_supprimer`

Pour supprimer une branche locale :

- `git branch -d nom_branche_a_supprimer`

### Bonnes pratiques pour la collaboration

- Synchroniser les branches avant de commencer à modifier le code, afin de travailler à partir de la version la plus récente.
- Effectuer des changements de petite taille et fréquents, plutôt que de gros changements, afin de limiter les risques de conflits.
- Fusionner régulièrement les modifications réalisées sur les branches secondaires dans la branche principale.
- Conserver une version stable du projet sur la branche principale et utiliser une ou plusieurs branches annexes pour les versions de test ou bêta.
- avoir de bon message de commit. Mettre beaucoup de détails surtout lorsque on est a plusieurs sur un projet.



## IV Collaboration

### 1 Pull Requests

Une *pull request* est un mécanisme de collaboration sur GitHub qui permet de proposer des modifications (commits) à un dépôt, même sans disposer des droits d'écriture sur celui-ci, afin que les responsables du projet puissent examiner les changements, en discuter et éventuellement les fusionner dans le code principal.

#### Une pull request simple sur GitHub — via l'interface web

GitHub permet de collaborer sur des projets en proposant des modifications à des dépôts auxquels on n'a pas accès en écriture. Le processus se déroule généralement comme suit :

- Créer une *fork* du dépôt original si l'on n'a pas les droits d'écriture.
- Modifier les fichiers concernés (via l'interface web ou localement).
- Enregistrer les changements sous forme de *commits* sur une nouvelle branche.
- Cliquer sur le bouton *Proposed File Change* pour créer un commit dans la *fork*.
- Compléter une description des changements.
- Ajouter des commentaires en ligne (*inline comments*) si nécessaire pour expliquer les modifications.
- Cliquer sur *Pull Request* pour proposer les modifications au dépôt original.

Le propriétaire du dépôt examine ensuite la *pull request* et décide de la fusionner ou non après vérification du code, des règles du projet et des éventuels conflits.

#### Le workflow typique d'une pull request sur GitHub — en local

Lorsque l'on souhaite travailler localement sur un projet distant, le workflow est le suivant :

- Créer une *fork* du dépôt distant sur son compte GitHub.
- Cloner la *fork* en local avec `git clone`.
- Modifier les fichiers localement.
- Enregistrer les changements avec des *commits*.
- Envoyer les modifications vers GitHub avec `git push`.
- Créer une *pull request* depuis l'interface GitHub pour proposer les changements au dépôt original.

Ce workflow permet à n'importe quel développeur de contribuer à un projet GitHub de manière structurée, même sans droits d'écriture sur le dépôt original.

#### Mettre à jour une pull request existante

Une *pull request* peut être mise à jour tant qu'elle n'a pas été fusionnée ou fermée. Pour cela, il suffit d'ajouter de nouveaux commits sur la même branche utilisée pour créer la *pull request*.

Le processus est généralement le suivant :

- Modifier à nouveau les fichiers concernés (en local ou via l'interface web).
- Créer de nouveaux *commits* avec les corrections ou améliorations demandées.
- Envoyer les commits vers la branche associée à la *pull request* avec `git push`.

GitHub met alors automatiquement à jour la *pull request* existante, sans qu'il soit nécessaire d'en créer une nouvelle. Cela permet de corriger des erreurs, de répondre aux commentaires des responsables du projet et d'améliorer progressivement la proposition avant sa fusion.

## Fusionner et combiner les commits (Squash)

Lorsqu'on souhaite fusionner une pull request, on peut choisir de **\*\*regrouper plusieurs commits en un seul\*\*** pour simplifier l'historique :

- **Merge commits** : tous les commits de la branche de fonctionnalités sont ajoutés à la branche principale dans un commit de merge (`-no-ff`).
- **Squash and merge** : tous les commits sont combinés en un seul commit. Le message de commit peut être modifié pour fournir une description complète.
- **Rebase and merge** : les commits sont appliqués individuellement sur la branche principale sans créer de commit de merge.
- **Indirect merges** : GitHub fusionne automatiquement si la branche a déjà été intégrée dans la branche principale.

**Squash interactif avec git rebase** Pour combiner des commits localement avant de pousser :

- Exécuter la commande : `git rebase -i master`
- L'éditeur affiche la liste des commits. Les actions possibles :
  - **pick** : conserver le commit.
  - **squash** : fusionner le commit avec le précédent et combiner les messages.
  - **fixup** : fusionner le commit avec le précédent en ignorant son message.
  - Modifier un commit pour changer son contenu ou son message.
- Enregistrer et quitter l'éditeur pour appliquer le rebase.
- Vérifier le dernier commit avec `git show` et l'état du dépôt avec `git status`.
- Si les anciens commits ont été publiés, forcer le push avec : `git push -f`. Cela permet de ne pas avoir de fusion avec les anciens commit mais juste de remplacer les anciens commits par les nouveaux.
- L'option `-f` (force) remplace l'historique distant par votre version locale. Cela permet d'éviter la création d'un commit de fusion inutile et de garder un historique propre.

Cette méthode permet de créer un commit unique avec un message complet, de simplifier l'historique et de rendre la pull request plus lisible pour les responsables du projet.

## 2 Code Reviews

Une *review de code* consiste à examiner le code, la documentation ou la configuration d'un autre développeur pour garantir sa qualité, sa cohérence et sa clarté.

**Objectifs :**

- Améliorer la qualité et la lisibilité du code.
- Détecter erreurs, oublis et incohérences.
- Diffuser les connaissances entre développeurs.

**Modes :**

- En personne ou à distance via des outils comme GitHub.
- Commentaires en ligne pour suggérer des améliorations.

## The code review workflow

Après avoir terminé des changements, le code est soumis à un réviseur qui peut ajouter des commentaires pour :

- Corriger des erreurs ou des oublis.
- Ajouter des tests manquants.
- Améliorer la clarté et la lisibilité du code.
- Suggérer des améliorations de style (Nits).

Le développeur corrige les problèmes, marque les commentaires résolus et peut répondre aux questions si nécessaire. Une fois tous les commentaires traités et le code approuvé, il peut être fusionné dans le dépôt principal.

### Conseils :

- Suivre un guide de style pour limiter les allers-retours.
- Traiter chaque commentaire comme une opportunité d'améliorer le code et sa documentation.
- L'objectif est d'assurer que le code est clair, cohérent et de haute qualité.

## Comment utiliser le code reviews

Sur GitHub, le processus de review de code inclut :

- Le réviseur ajoute des commentaires globaux et ligne par ligne sur la pull request.
- L'auteur corrige les commentaires : fautes de frappe, mise en forme, ajout d'exemples, etc.
- Les changements peuvent être inclus dans le commit précédent avec `git commit --amend`.
- Les commits amendés nécessitent un `git push -f` pour mettre à jour la pull request.
- Les commentaires résolus sont marqués comme tels et l'auteur peut notifier le réviseur.
- Une fois satisfait, le réviseur approuve les changements et la pull request peut être fusionnée.

### Conseils :

- Traiter chaque commentaire comme une opportunité d'améliorer le code.
- Pratiquer régulièrement les reviews pour maîtriser le processus.
- Utiliser les guides de style et bonnes pratiques pour réduire les allers-retours.

## 3 Managing Projects

### Tracking issues

La coordination est essentielle lors du travail collaboratif afin d'éviter les doublons et les oublis. Les *issue trackers* permettent d'organiser le travail, en particulier lorsque les équipes grandissent.

Les *issues* servent à :

- Lister les bogues, tâches et fonctionnalités à implémenter.
- Indiquer l'état d'avancement et la personne responsable.
- Centraliser les discussions, solutions et méthodes de test.
- Permettre aux utilisateurs de signaler des problèmes.
- Aider les nouveaux contributeurs à identifier où contribuer.

## GitHub Issues

- Intégré nativement à GitHub.
- Chaque *issue* et *pull request* possède un identifiant unique.
- Les références se font avec **#numero** (ex : **#2**).

## Création et gestion

- Titre clair et description détaillée du problème ou de la fonctionnalité.
- Ajout d'informations techniques et d'idées de résolution.
- Possibilité d'assigner une *issue* à un collaborateur.

## Lien entre Issues et Pull Requests

- Une *issue* peut être fermée automatiquement lors du merge.
- Utilisation de mots-clés comme **closes #1**, **fix #1** dans un commit ou une PR. (dans le message de commit plus précisément)
- GitHub ferme l'issue lors de l'intégration du code.

Un suivi des problèmes permet de lister les problèmes/bug

## Intégration Continue et Déploiement Continu (CI/CD)

Tester manuellement le code est peu fiable. Pour automatiser ce processus, on utilise des tests automatisés couplés à des systèmes de CI/CD.

### Intégration Continue (CI)

- Exécute automatiquement la compilation et les tests.
- Se déclenche à chaque commit sur la branche principale.
- S'applique aussi aux *pull requests*.
- Garantit que le code reste fonctionnel après chaque changement.

### Déploiement Continu (CD)

- Déploie automatiquement le code après validation.
- Favorise des mises à jour fréquentes et incrémentielles.
- Permet de détecter et corriger les erreurs plus tôt.

## Pipelines

- Définissent les étapes à exécuter (build, tests, déploiement).
- Varient selon le projet et la technologie utilisée.

## Artefacts

- Fichiers générés par le pipeline (binaires, paquets, documentation).

## Sécurité et Secrets

- Gestion sécurisée des accès (clés SSH, tokens API).
- Séparer les accès test et production.
- Toujours prévoir un plan de récupération en cas de compromission.

## Outils CI/CD

- Jenkins, GitLab CI, Travis CI.
- Travis CI s'intègre avec GitHub via un fichier `.travis.yml`.

## 4 Glossary terms

- CI/CD: The name for the entire continuous integration and continuous deployment system
- Code reviews: The deliberate and methodical gathering of other programmers to examine each other's code for errors to increase the code quality and reduces the amount of bugs
- Continuous deployment (CD): New code is deployed often after it has been automatically built and tested
- Continuous integration (CI): A system that will automatically build and test our code every time there's a change
- Fix up: The decision to discard commit messages for that commit
- Forking: A way of creating a copy of the given repository so that it belongs to our user
- Indirect merges: GitHub can merge a pull request automatically if the head branch is directly or indirectly merged into the base branch externally
- Issue tracker (bug tracker): A tracker that shows tasks that need to be done, the state they're in and who's working on them
- Merge commits: All commits from the feature branch are added to the base branch
- Pipelines: The specific steps that need to run to obtain the desired result
- Pull request: A procedure where new code is examined before it is merged to create a branch or master branch
- Squash commits: The decision add commit messages together and an editor opens to make any necessary changes